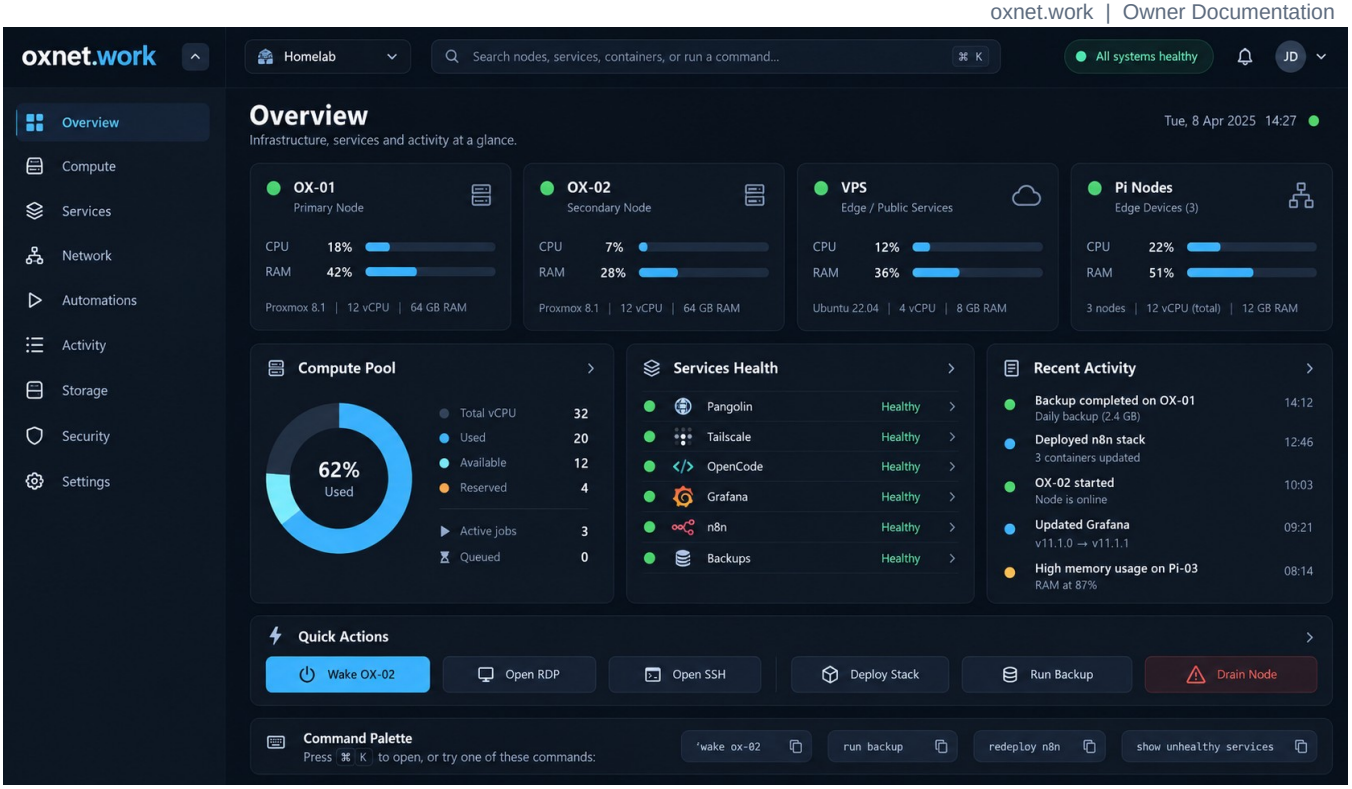


OWNER PROJECT DOCUMENTATION

oxnet.work

Private command centre for access, compute, monitoring and automation

Build specification edition - 17 October 2026

Document purpose

This document is the owner-facing explanation of the agreed oxnet.work product, architecture, interface, security model and implementation roadmap. The development-agent handoff is supplied separately in the project pack.

Contents

1	Executive summary
2	What oxnet.work is - and is not
3	Approved platform architecture
4	Lightweight deployment model
5	Command Centre UI
6	Compute and OpenCode model
7	Network and access model
8	Automation model
9	Security and trust boundaries
10	Integration plan
11	Build roadmap
12	Owner operations and recovery
13	Agent development model
14	Decisions and future direction

Important

Values shown inside the UI mockups are illustrative design data. They must not be treated as the owner's live host specifications, IP addresses, software versions, usernames, or capacities.

1. Executive summary

oxnet.work will be a small self-hosted control plane that brings together the systems already forming the OXNetwork: private connectivity, controlled remote exposure, remote administration, Docker services, shared OpenCode compute, monitoring, automation and recovery. Its job is not to replace those specialist systems; its job is to make them understandable and operable from one coherent interface.

- One command centre: health, capacity, services, jobs, topology, automations and activity in one UI.
- Lightweight by design: one Go control-plane process, compiled React frontend, SQLite, optional tiny node agents.
- Private by default: use Tailscale for normal access; Pangolin exposure is optional and protected by authentication/access policy.
- Safe actions: wake, drain, restart, deploy and backup are capability-scoped, confirmed and audited.
- No heavyweight orchestration platform is required: no Redis, Kafka, Kubernetes or external database for v1.
- OpenCode becomes part of a visible compute pool rather than a separate collection of machines.

Decision	Approved direction
Project/domain	oxnet.work
Primary role	Private infrastructure command centre
Default runtime	Single Docker control-plane container + optional node agents
Backend	Go
Frontend	React + TypeScript + Vite
Data store	SQLite with migrations and WAL
Live updates	Server-Sent Events
Default access	Tailscale/private network

2. What oxnet.work is - and is not

The layer above the tools

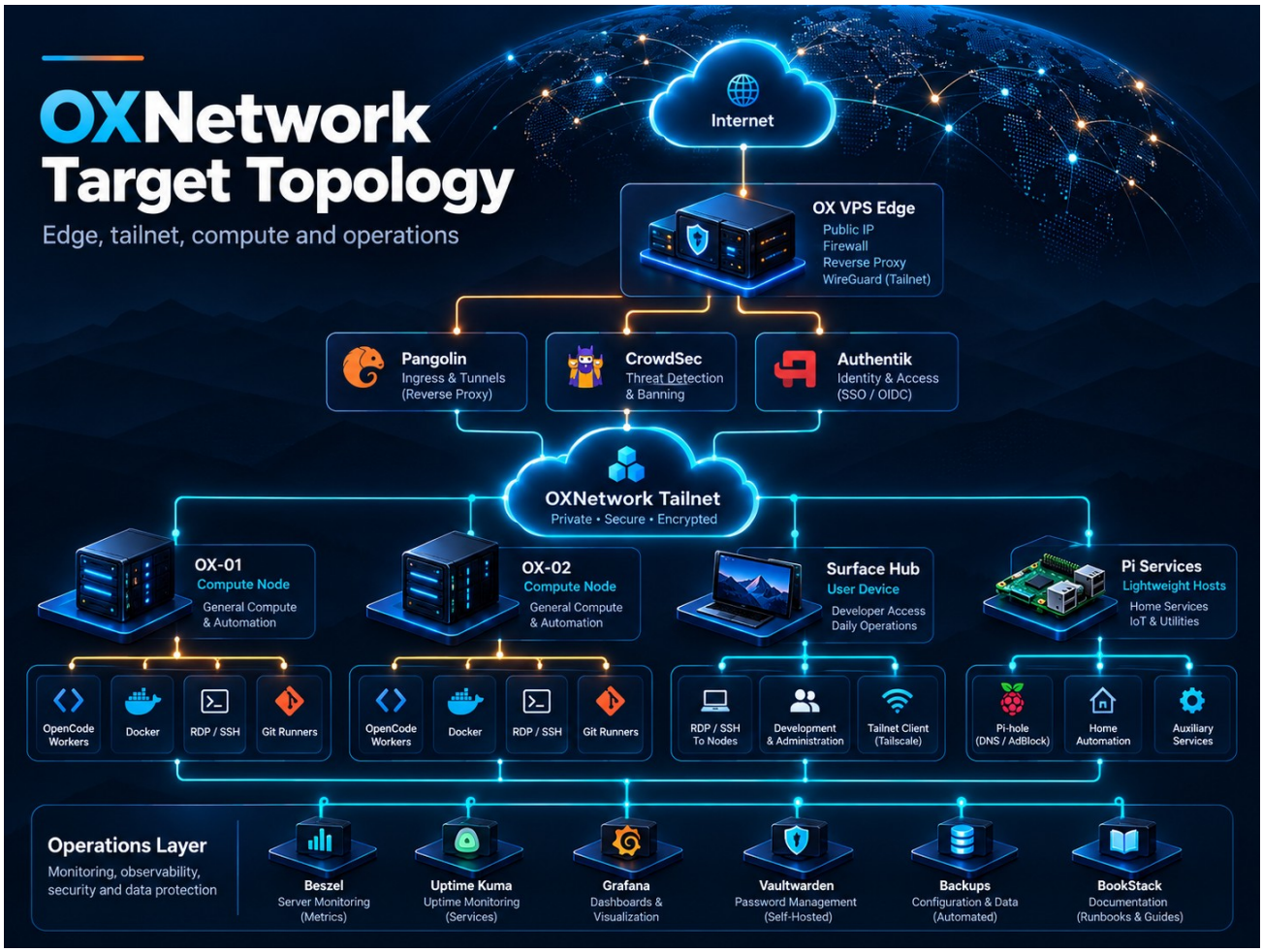
Pangolin, Tailscale, Docker, Grafana, monitoring tools and OpenCode each solve specialist problems. oxnet.work sits above them as the owner-facing operational layer: it normalizes their state, shows relationships, exposes a small number of safe actions and records what happened.

Existing capability	How oxnet.work complements it
Pangolin	Controlled external access / exposed resources; oxnet.work shows exposure and links/actions where safe.
Tailscale	Private device mesh and identity/reachability; oxnet.work uses it for node context and private access.
RDP / SSH	Direct administration; oxnet.work launches/copies connection details rather than embedding a full remote terminal in v1.
Docker	Container runtime; oxnet.work inventories and later performs bounded actions through local agents.
OpenCode	Distributed coding/compute; oxnet.work presents workers, jobs, capacity and placement.
Monitoring	Deep metrics remain in specialist tools; oxnet.work shows operational summaries and freshness.
Automation	oxnet.work provides simple, auditable operational workflows, not a general enterprise workflow engine.

Explicit v1 boundaries

- No arbitrary browser shell.
- No Kubernetes requirement.
- No full SIEM/log warehouse.
- No complex drag-and-drop workflow canvas.
- No mandatory cloud/SaaS dependency.
- No public exposure until authentication/security hardening is complete.

3. Approved platform architecture



Conceptual target topology. The final application will use the restrained production UI style rather than the presentation-style neon treatment.

Control plane

The central oxnet.work service is deliberately small. One Go process hosts the REST API, SSE updates, authentication/session handling, scheduler, integration polling, audit recording, SQLite access and the compiled web UI. This minimizes container count and reduces operational overhead.

Node agents

Optional Go agents run only on hosts where local metrics, Docker inventory or privileged actions are required. Agents communicate over the private network and expose typed capabilities rather than a generic shell. A node can still exist in the inventory without an agent if it is discovered through Tailscale or another provider.

Why this architecture

The aim is to create a genuine control plane without creating another large platform that itself needs substantial maintenance. Integration boundaries remain replaceable so the architecture can grow later without forcing v1 to carry enterprise infrastructure.

4. Lightweight deployment model

Default deployment

Area	Target
Control plane	1 container
Persistent data	1 volume containing SQLite and local app state
External database	Not required
Message broker	Not required
Realtime transport	SSE
Docker access	Via node agent - not by mounting Docker socket into the public control plane
CPU at idle	Target near-zero outside polling
Control-plane memory	Engineering target below roughly 150 MB idle
Agent memory	Engineering target below roughly 30 MB idle

Why Go + static frontend

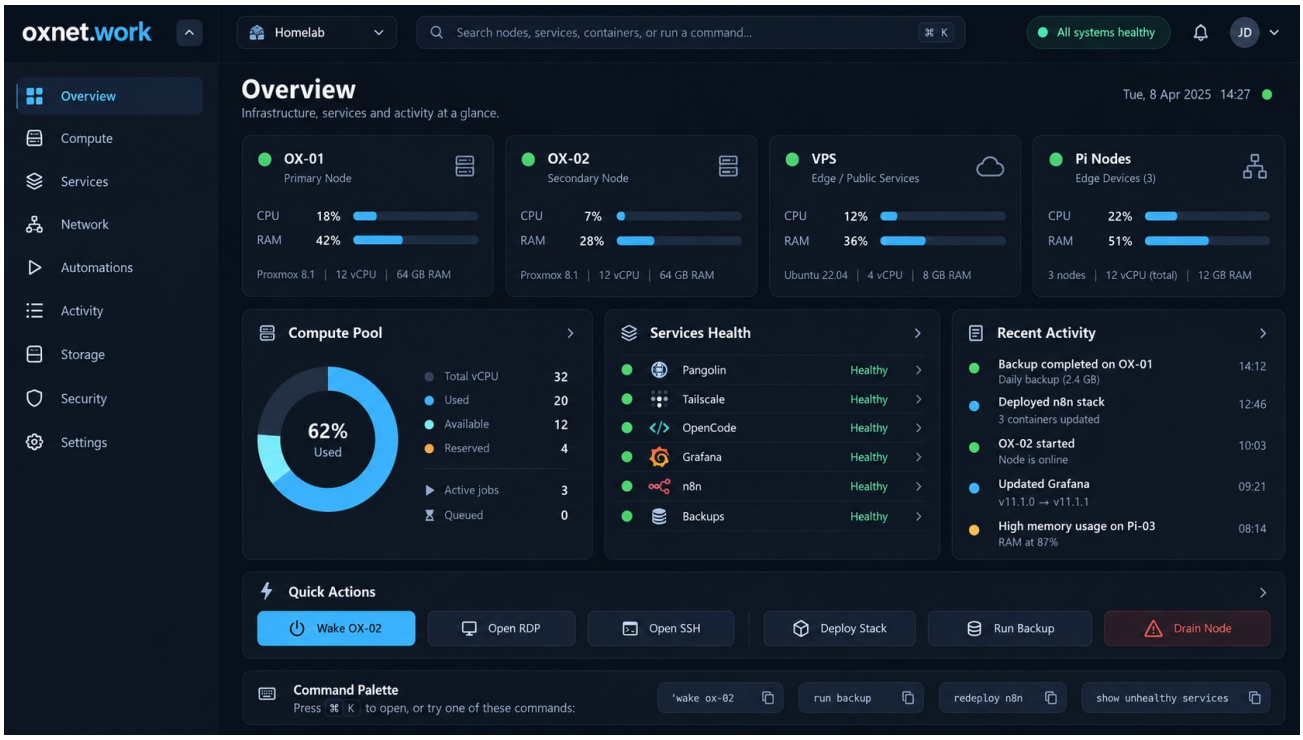
A Go server and small compiled React frontend give the project a rich UI while keeping the production runtime simple. The browser receives static assets; the server does not need a Node.js runtime in production. SQLite removes a database service and associated maintenance burden.

Resource targets are targets, not guarantees

Codex should measure these values during implementation. A dependency that materially increases idle footprint or browser bundle size should be justified in the decision log.

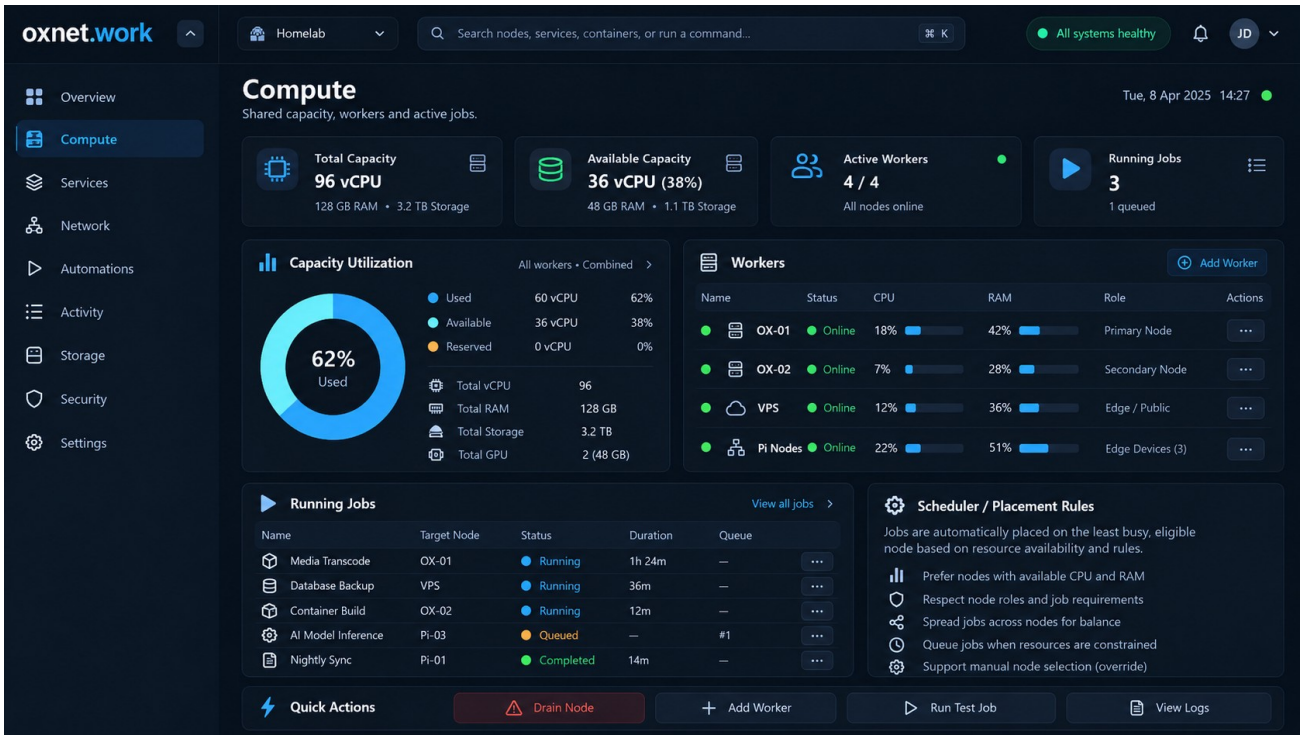
5. Command Centre UI

The approved interface is intentionally calmer than the earlier presentation artwork. The product should feel like mission-control capability with Linear/Vercel restraint: dark, information-dense, clean, responsive and buildable with normal web components.

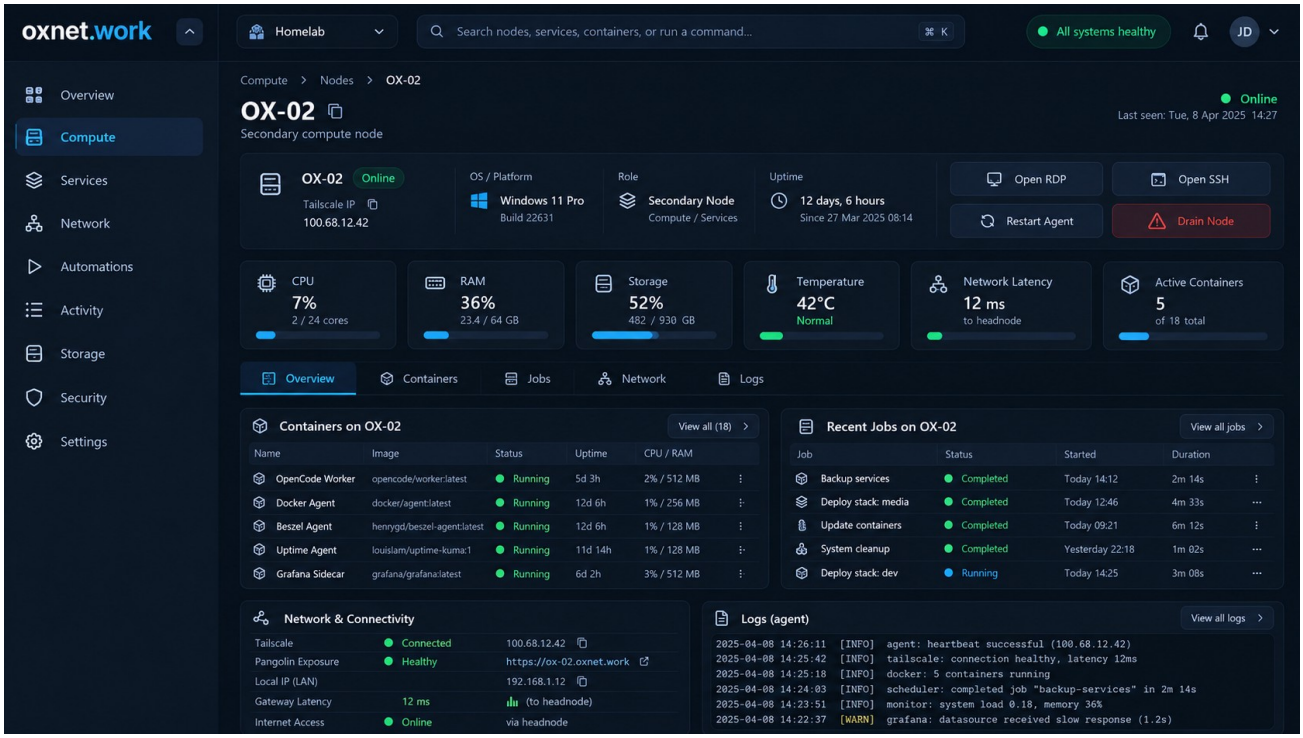


Approved Overview direction: compact node health, compute pool, service state, recent activity and capability-aware actions.

5.1 Compute and node detail

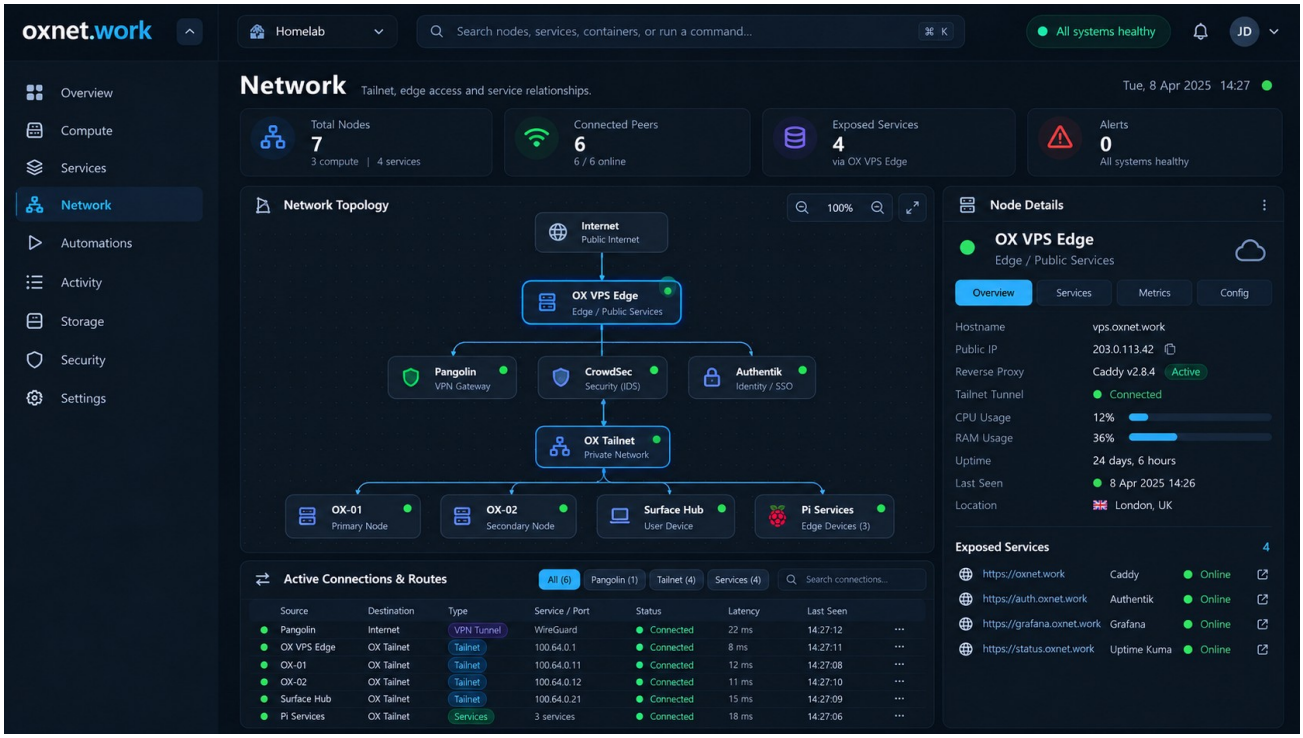


Compute screen: capacity, workers, jobs and deterministic placement rules.

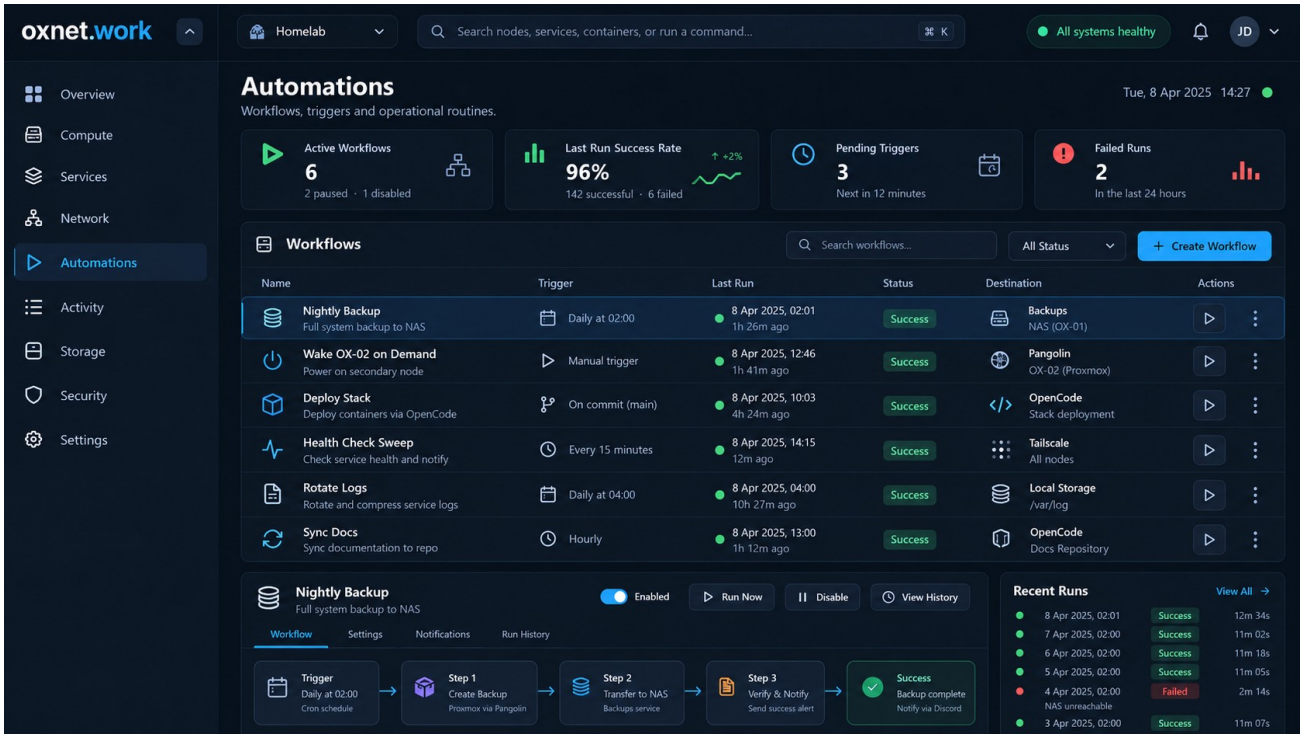


Node detail: identity, live metrics, containers, recent jobs, network state and bounded log preview.

5.2 Network and automations



Network view: a simple SVG topology with a selected-node detail panel and a connection/route table.



Automations view: table-first workflow management with a lightweight linear step diagram.

6. Compute and OpenCode model

oxnet.work should treat distributed compute as a pool while still respecting the limits of individual machines. Aggregate vCPU/RAM numbers are informational; a job can only run on a worker that actually satisfies its requirements.

Placement process

1. Filter to online, fresh, non-drained workers.
2. Filter by capabilities, architecture, tags and GPU requirements.
3. Exclude workers that fail minimum free CPU/RAM/GPU thresholds.
4. Apply explicit affinity and exclusion rules.
5. Score the remaining workers using normalized free resources/load.
6. Choose deterministically and record why the worker was selected.

No AI required for placement

The v1 scheduler should be deterministic and explainable. AI-assisted decisions can be explored later, but the underlying rule-based placement must remain available and auditable.

7. Network and access model

Preferred access path

Normal owner access should be through Tailscale/private connectivity. The control plane does not need to be internet-facing to be useful.

Optional Pangolin path

If the owner later wants oxnet.work reachable outside the tailnet without installing Tailscale on the client device, Pangolin can publish it. That should happen only after authentication, authorization, session hardening and access policy are complete.

Capability	Owner model
Tailscale	Private connectivity and node/tailnet discovery context.
Pangolin	Controlled external resource exposure and edge inventory.
RDP	Launch/copy connection workflow; credentials are not stored in plaintext by oxnet.work.
SSH	Copy/open connection details; no arbitrary browser shell in v1.
Wake-on-LAN	Executed by an appropriate LAN-side agent/proxy when the target is not reachable from the VPS broadcast domain.

8. Automation model

Automations are intended for infrastructure routines: backups, health sweeps, wake-on-demand, deployment steps, log rotation, synchronization and notifications. The design is ordered and explicit rather than a freeform flow-canvas.

Supported v1 trigger types

- Manual trigger
- Cron/scheduled trigger
- Simple health/event condition trigger

Step design

- Typed action step against a known target/provider
- Delay/wait where required
- Notification step
- Stop or continue-on-error policy
- Success/failure outcome and audit record

Key owner benefit

Routine infrastructure work becomes visible and repeatable. A workflow run produces a step-by-step history instead of disappearing into an untracked script or terminal session.

9. Security and trust boundaries

oxnet.work will eventually be able to perform privileged actions, so its security model must be stronger than a normal dashboard. The design intentionally avoids a general remote-command interface and keeps privileged capabilities close to the node through scoped agents.

Boundary	Required approach
Authentication	OIDC-ready; Authentik preferred, provider-neutral implementation.
Browser session	Secure HttpOnly SameSite cookies, expiry, rotation and CSRF protection.
Authorization	Permission scopes for read, safe actions, destructive actions, workflows, integrations and security.
Secrets	Environment/mounted secret references first; plaintext integration secrets must not live in ordinary SQLite fields.
Agent trust	Unique agent identity, scoped credentials, typed capabilities/actions and action IDs.
Audit	Append-only operational event trail containing actor, target, action and result.
Logs	Secrets, auth headers and environment dumps must be redacted/never logged.

High-impact action rule

Restart, shutdown, destructive container changes, restore, or drain-with-active-work must show the target and impact, require explicit confirmation and create an audit record. The UI should never make destructive actions visually casual.

10. Integration plan

Live integrations are deliberately phased. The UI and domain model must work first against deterministic fixtures so development is not blocked by access to every live system.

Order	Integration
1	Demo/fixture provider - proves UI and domain contracts.
2	Tailscale discovery - nodes, addresses, reachability context.
3	Docker read-only via agent - containers and state.
4	Host metrics - Beszel and/or direct agent metrics.
5	Uptime Kuma - operational service state where useful.
6	Pangolin - exposed-resource inventory and relationships.
7	OpenCode - workers and jobs.
8	Wake/node actions and backups - typed, auditable actions.
9	Authentik/OIDC - hardened authentication.
10	Optional Grafana/Prometheus deep links/query adapters.

Graceful degradation

A broken or unavailable integration must not take down the rest of the interface. oxnet.work should preserve last-known state, show freshness, and make the failure obvious.

11. Build roadmap



Visual roadmap concept. The implementation roadmap below is the authoritative build order.

Milestone	Outcome
M0	Repository, Go/Vite shell, SQLite, config, Docker, CI.
M1	Demo vertical slice and Overview.
M2	Compute and Node Detail.
M3	Network topology.
M4	Automations and audit.
M5	Node agent foundation.
M6	Tailscale + Docker read-only.
M7	Pangolin exposure inventory.
M8	OpenCode live compute.
M9	Safe actions.
M10	OIDC/security hardening.
M11	Backup/restore, polish and owner release.

12. Owner operations and recovery

Routine checks

- Global health and stale/offline nodes.
- Failed services or workflows.
- Available compute and active/queued jobs.
- Recent changes/actions.
- Backup freshness and last successful run.

Maintenance pattern

- Drain a compute node before maintenance.
- Confirm active jobs are complete or intentionally handled.
- Run the maintenance action.
- Verify the node returns healthy and becomes schedulable.
- Review the audit/activity event.

Recovery priority

1. Restore private/control-plane access.
2. Restore SQLite/config from a verified backup.
3. Restore secret sources or reconfigure integrations.
4. Start read-only if action providers are unavailable.
5. Re-enrol agents only when credentials cannot safely be recovered.

Backup principle

A green backup status means the backup job ran successfully. It does not prove recoverability. The project roadmap therefore includes a documented test-restore procedure.

13. Agent development model

Codex

Codex is the primary implementation and architecture agent. It owns the Go control plane, database, API, node-agent trust/protocol, integration architecture, compute/scheduler behavior, security-sensitive backend actions, release packaging and integration of other contributions.

Claude Code

Claude Code can be used as a supporting implementation agent for bounded work packages that have clear interfaces. The supplied CLAUDE.md assigns good candidates: reusable frontend components, SVG topology rendering, typed frontend API client, workflow UI and deterministic Playwright fixtures/tests.

Work area	Default owner
Frontend component library	Claude candidate
SVG topology renderer	Claude candidate
Workflow visual editor/viewer	Claude candidate
Playwright UI coverage	Claude candidate
Authentication architecture	Codex only unless explicitly reassigned
Agent enrollment/trust	Codex only
Scheduler semantics	Codex only
Destructive backend actions	Codex only

Integration rule

Codex reviews and integrates Claude Code output. Claude contributions do not override AGENTS.md or the architectural decision log.

14. Decisions and future direction

Locked for the initial build

- Project/domain name: oxnet.work.
- Lightweight one-service control-plane default.
- Go backend and node agent.
- React + TypeScript + Vite frontend.
- SQLite with migrations/WAL.
- SSE for live dashboard updates.
- SVG topology, no heavy 3D/WebGL.
- Typed agent actions, no arbitrary shell.
- Private/Tailscale-first access.
- Approved five-screen Command Centre UI direction.

Deliberately deferred

- AI natural-language operations in the command palette.
- Highly available multi-control-plane mode.
- Advanced distributed queue/broker.
- Kubernetes orchestration.
- Public-facing multi-user SaaS features.
- Large-scale historical metrics/log storage.

Owner decision points during implementation

- Where the first production control plane will run.
- Which hosts receive the oxnet agent first.
- Final Authentik/OIDC exposure policy.
- Which backup repository/tool becomes the first supported provider.
- Exact OpenCode worker/job interface once the shared-compute deployment is final.
- Whether oxnet.work remains tailnet-only or is later published through Pangolin.

North star

A private cloud command centre that can explain its own health, show where capacity exists, automate routine work, and route approved compute/jobs to the right node - without becoming heavyweight infrastructure itself.

Development handoff supplied separately: AGENTS.md, CODEX.md, CLAUDE.md and supporting specifications.